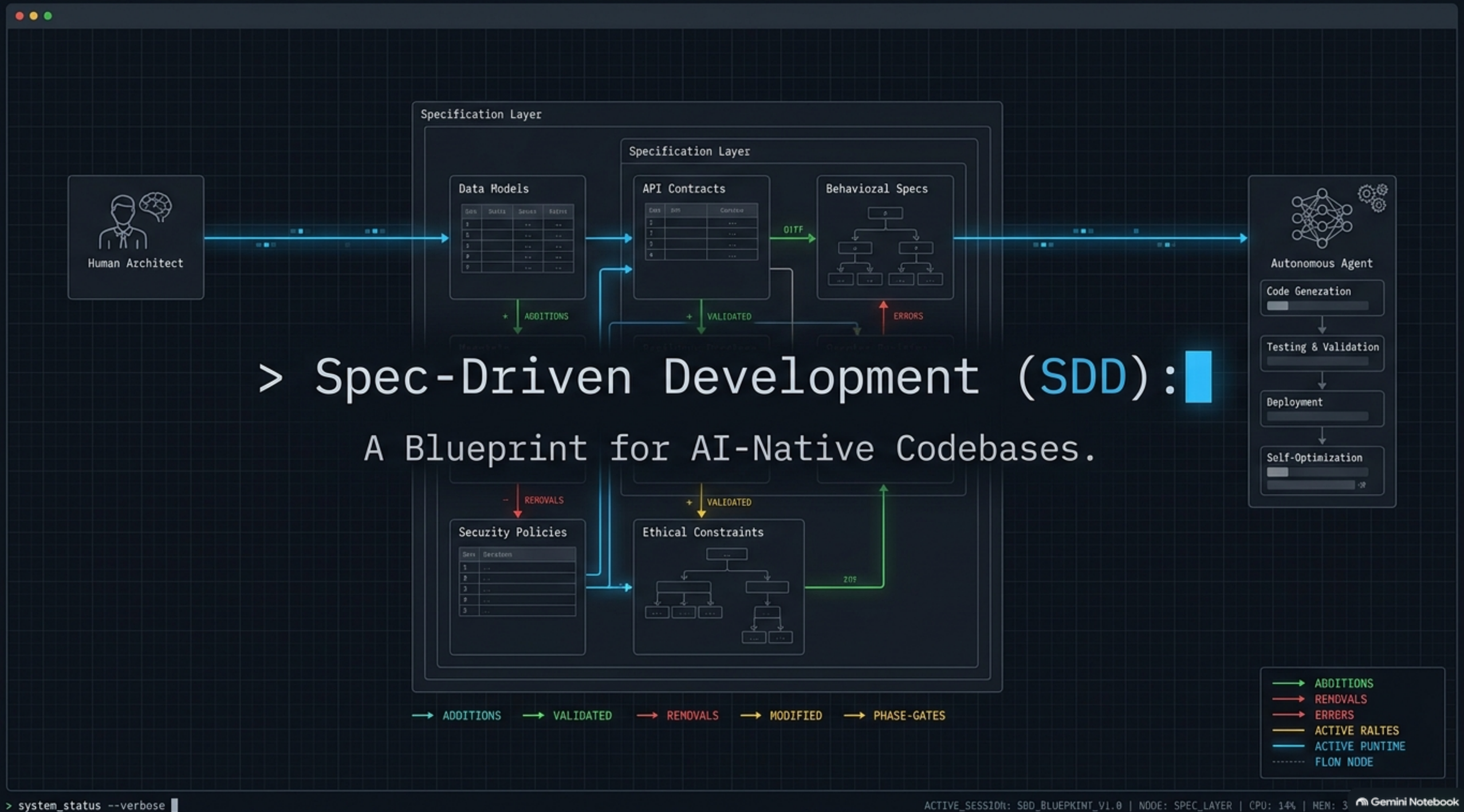

Specification Layer
Specification Layer
Human Architect
Data Models
API Contracts
Behaviozal Specs
Autonomous Agent
Code Genezation
Testing & Validation
Deployment
Self-Optimization
+ AGOITIONS
+ VALIOATED
ERRORS
OITF
> Spec-Driven Development (SDD):
A Blueprint for AI-Native Codebases.
- REMOVALS
+ VALIOATED
Secuzity Policies
Ethical Constraints
ZOF
ADDITIONS
VALIDATED
REMOVALS
MODIFIED
PHASE-GATES
ABOITIONS
RENOVALS
ERRORS
ACTIVE RALTES
ACTIVE PUNTIME
FLON NODE
> system_status --verbose
ACTIVE_SESSIOn: SBD_BLUEPKINT_V1.0 | NOOE: SPEC_LAYER | CPU: 14% | MEN: 3
Gemini Notebook

001

THE LIMITS OF CHAT

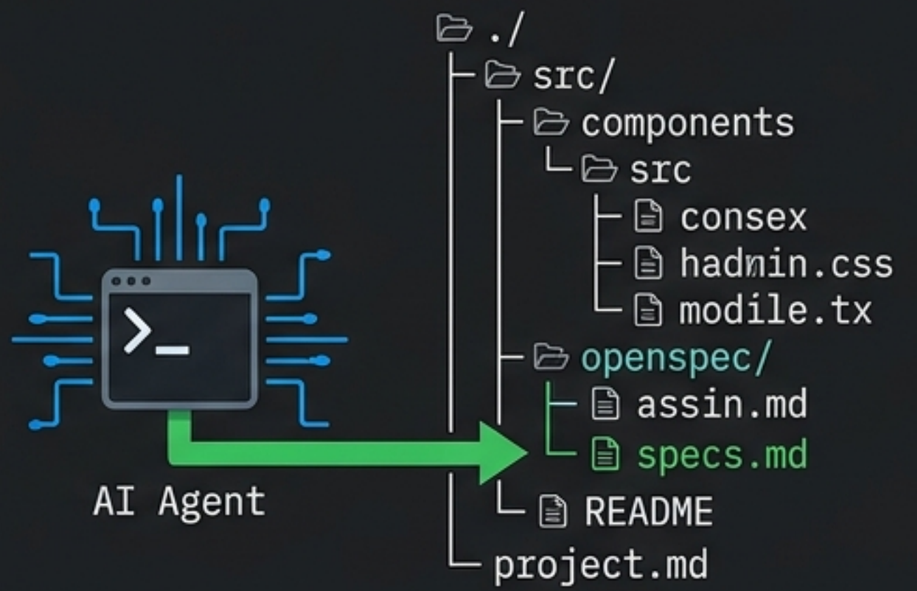
002



002

Without durable constraints, AI assistants fall into requirement hallucination, non-deterministic choices, and silent regressions.

THE SDD PARADIGM



Spec-Driven Development shifts design from ephemeral prompting to version-controlled, machine-readable agreements.

003

> As Andrej Karpathy notes, “programming as vibing with AI” hits a hard wall at scale; without durable constraints, even 1M token context windows exhaust into architectural drift.

Gemini Notebook



Unstructured AI Chat

- Ephemeral prompts
- Rapid prototyping
- Highly unpredictable at scale

Task-Centric AI

[TDD, CI/CD quality gates, Automated E2E test scripts]

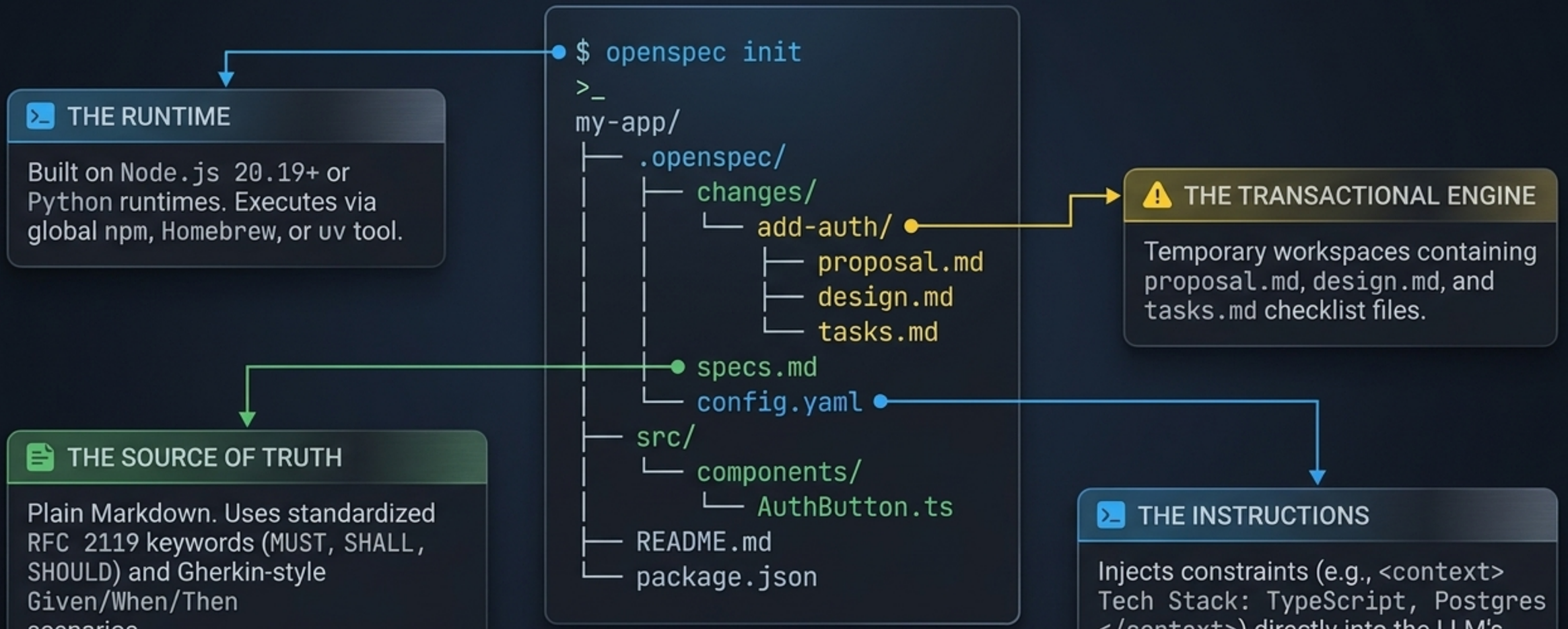
Spec-Driven Development

[Version-controlled Markdown specs, ADRs, Multi-model adversarial review]

As AI autonomy scales, reliance must shift from conversational 'vibes' to repository-anchored, durable constraints.

Gemini Notebook

ANATOMY OF AN SDD FRAMEWORK



THE RUNTIME

Built on `Node.js 20.19+` or `Python` runtimes. Executes via global `npm`, `Homebrew`, or `uv` tool.

THE TRANSACTIONAL ENGINE

Temporary workspaces containing `proposal.md`, `design.md`, and `tasks.md` checklist files.

THE SOURCE OF TRUTH

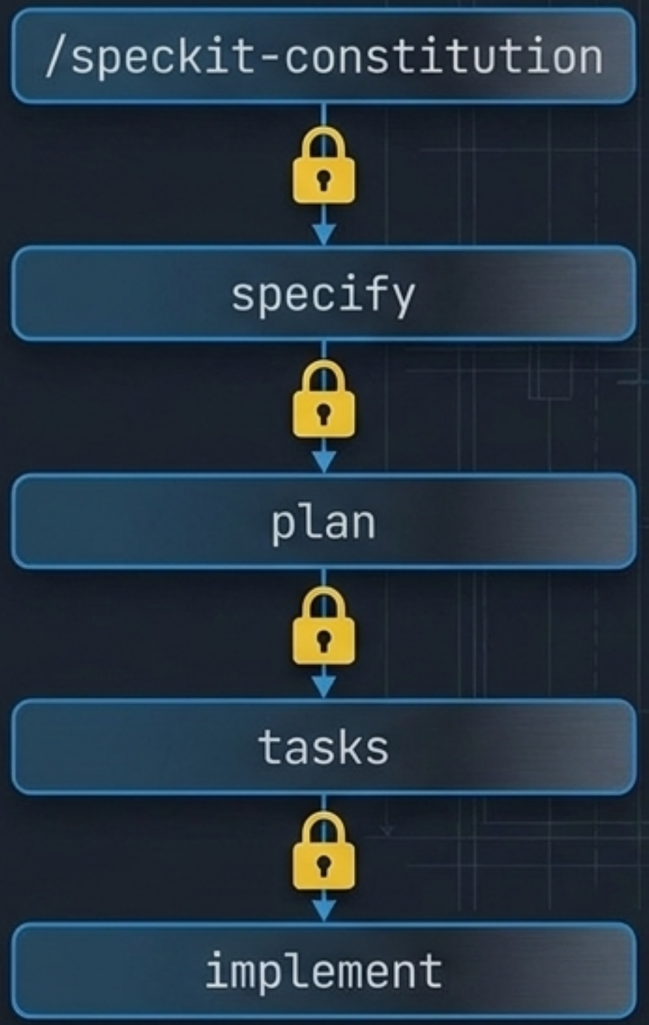
Plain Markdown. Uses standardized `RFC 2119` keywords (`MUST`, `SHALL`, `SHOULD`) and Gherkin-style `Given/When/Then` scenarios.

THE INSTRUCTIONS

Injects constraints (e.g., `<context> Tech Stack: TypeScript, Postgres </context>`) directly into the LLM's system prompt.

Gemini Notebook

> Framework_01: GitHub Spec Kit



Bias:

Greenfield initialization & multi-player readiness.

Under the Hood:

Python CLI (`uv tool install`).

The Constitution:

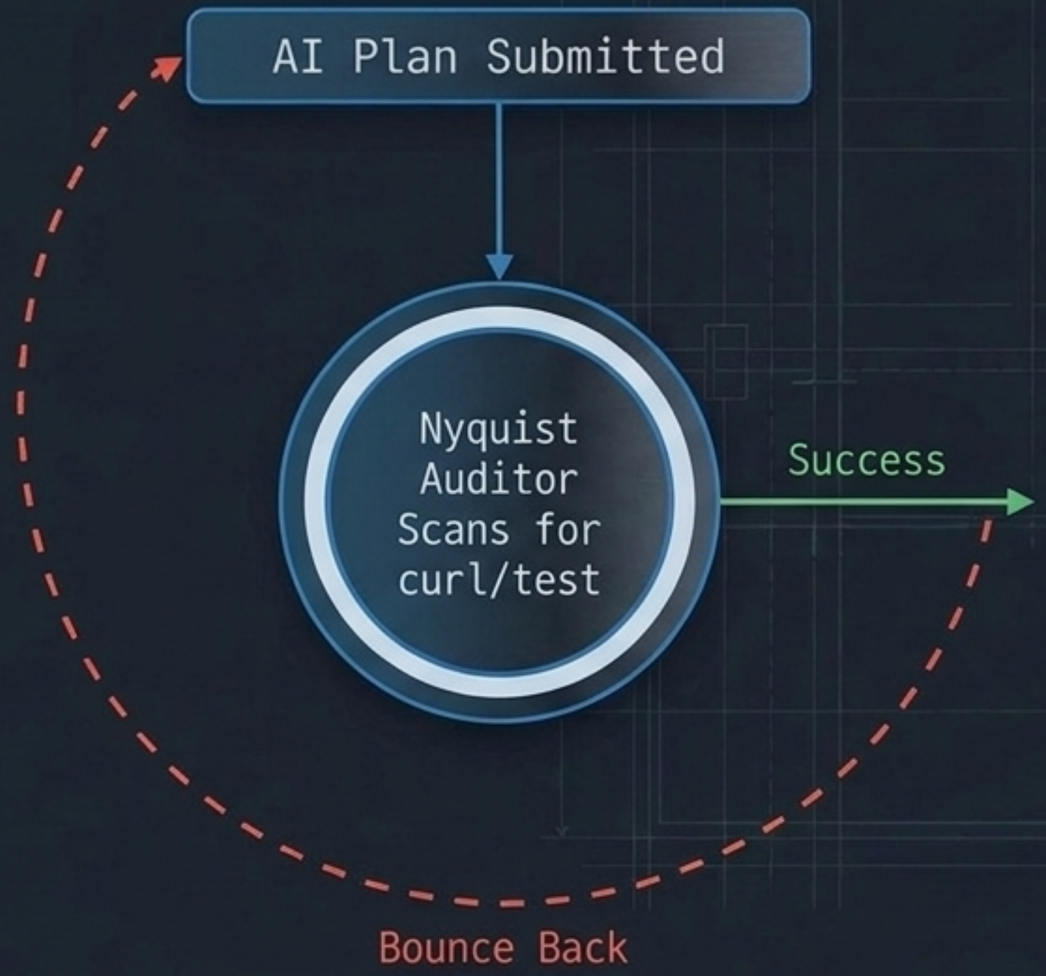
Uses a rigid `constitution.md` to enforce project-wide rules (e.g., Test-First mandates).

The Trade-off:

High overhead. Generates 8 files per spec. Excellent for formal architectures; overkill for a 3-point Jira ticket.

Gemini Notebook

> Framework_02: GSD (Get-Shit-Done)



Bias:

Atomic execution and context hygiene.

Under the Hood:

Lightweight shell scripts and terminal dependencies.

The Mechanism:

Bypasses formal specs for phase-based delivery. Emphasizes automated feedback loops over document generation.

The Trade-off:

Transitory logs. Does not produce a living, reviewable specification document that survives the implementation.

Gemini Notebook

> Framework_03: AWS Kiro



Integrated strictly into AWS IDEs. Locked to Claude model famiilies and cloud metadata stores.

Powerful, but lacks vendor neutrality.

> Framework_04: OpenSpec



The most adopted OSS framework (~8,000 public repos, 72k stars). Built on a Node.js CLI.

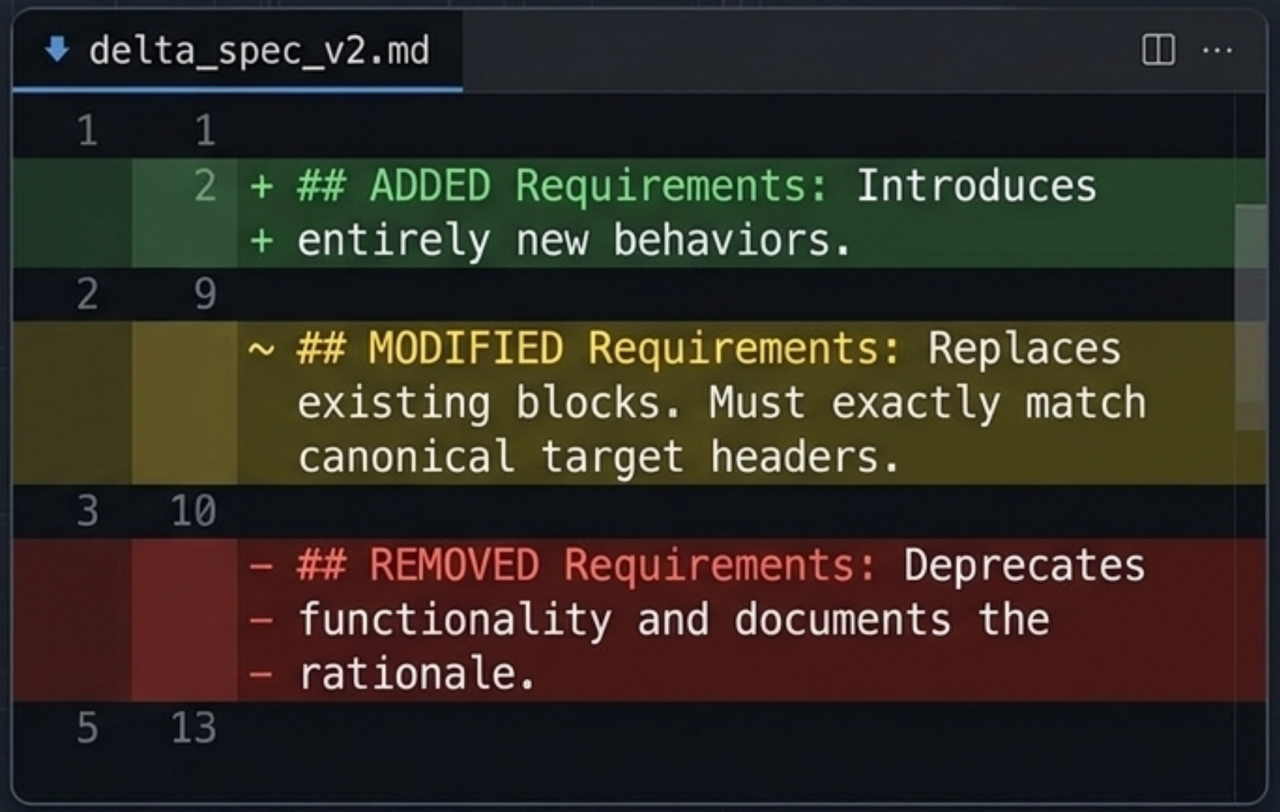
Integrates with 30+ tools via slash commands (e.g.,`/opsx:propose`). Vendor-agnostic, supporting Claude Code, Copilot, Cursor, etc.

Gemini Notebook

> The Brownfield Solution: Delta Specifications

You cannot write a 40-page spec for an undocumented 80,000-line legacy app.

OpenSpec circumvents this through differential mutations.



Deltas isolate in-flight changes. Once verified, they automatically merge into the openspec/specs/ living source of truth.

Gemini Notebook

The OPSX Workflow Lifecycle



Discovery

/opsx:explore

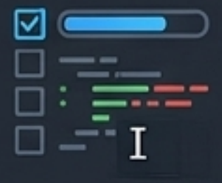
No-stakes codebase analysis. The agent maps dependencies without writing permanent artifacts.



Scaffolding

/opsx:propose

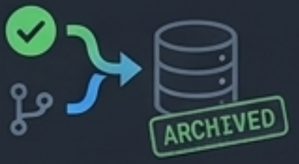
Instantiates the change workspace (openspec/changes/<id>/). Generates the proposal.md, delta specs/, design.md, and tasks.md.



Implementation

/opsx:apply

The AI agent processes the tasks.md checkboxes sequentially, executing code edits and generating unit tests.



Synchronization

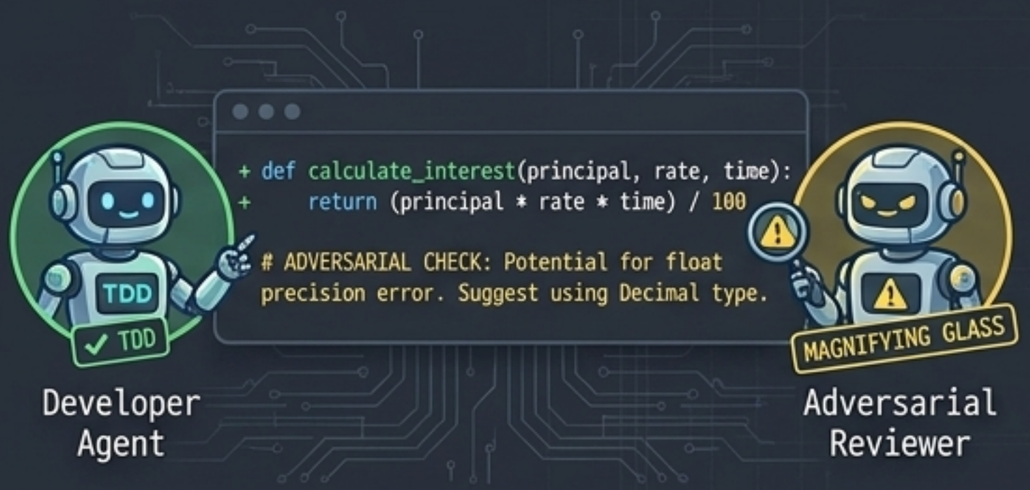
/opsx:archive

Validates task completion, merges delta specs into the living source of truth, and moves the folder to the historical archive.

Gemini Notebook

Beyond the Spec: Verification & Validation

The anvil Schema (TDD + Adversarial Review)



Enforces Test-Driven Development (TDD) alongside an adversarial review gate. A secondary, isolated LLM reviewer audits the plan before implementation is allowed.

The e2e-runbooks Schema

```
> [2024-10-26 10:45:00] RUNBOOK START: 'User Registration Flow'
> [2024-10-26 10:45:02] ACTION: POST /api/v1/users -
{"email": "test@example.com", "password": "secure"}
> [2024-10-26 10:45:03] VERIFY: HTTP 200 OK
✅
> [2024-10-26 10:45:03] ASSERTION: Database user record created
correctly.
✅
> [2024-10-26 10:45:05] RUNBOOK END: 'User Registration Flow' -
SUCCESS
```

Generates end-to-end execution runbooks with timestamped behavior logs and strict HTTP/database verification assertions.

Heavy evals and rigorous automated testing are the mandatory counterparts to AI code generation. Code cannot be generated safely without an isolated, adversarial verification loop.

Gemini Notebook

Master Framework Comparison

Architectural Dimension	OpenSpec	GitHub Spec Kit	GSD
Specification Model	Delta specs merging into unified living specs	Hierarchical docs with rigid phase gating	Lightweight execution scripts
Runtime Environment	Node.js CLI	Python CLI (uv)	Shell scripts
Codebase Bias	Optimized for brownfield and greenfield	Biased toward greenfield	Task-centric execution
Persistence Lifecycle	Deltas archived; Living specs persist	Static documents in dedicated subtrees	Transient task logs discarded post-implementation

Gemini Notebook

When to Use Which: The Decision Framework

Are you modifying an undocumented, brownfield legacy application (e.g., 80k lines of code)?

YES → Use **OpenSpec**. (Leverage differential specification without upfront documentation).

NO ↓

Is this a brand new, greenfield project requiring strict, formal phase-gating and multi-player team alignment?

YES → Use **GitHub Spec Kit**. (Leverage the constitution.md and hierarchical document trees).

NO ↓

Do you want to skip formal specification entirely and focus strictly on atomic task execution and context hygiene?

YES → Use **GSD**. (Leverage the Nyquist Auditor and shell-script automation).

The tool is not the answer. SDD frameworks cannot replace engineering judgment; they simply force that judgment to happen explicitly, before the agent touches a file.

Gemini Notebook